



第4章 串

提纲
CONTENTS

4.1 串的基本概念

4.2 串的存储结构

4.3 串的模式匹配



4.1 串的基本概念

串（或字符串）是由零个或多个字符组成的有限序列。

串 $\subset$ 线性表

- 串中所含字符的个数称为该**串的长度**（或串长）。
- 含零个字符的串称为**空串**，用 Φ 表示。



串的逻辑表示:



串的逻辑表示, a_i ($1 \leq i \leq n$) 代表一个字符。



串相等：两个串的长度相等并且各个对应位置上的字符都相同时。

如：

“*abcd*” $\neq$ “*abc*”

“*abcd*” $\neq$ “*abcde*”

所有空串是相等的。



子串：一个串中任意个连续字符组成的子序列（含空串）称为该串的子串。

例如，“*abcde*”的子串有：

“”、“*a*”、“*ab*”、“*abc*”、“*abcd*”和“*abcde*”等

真子串是指不包含自身的所有子串。



串抽象数据类型=逻辑结构+基本运算（运算描述）

串的基本运算如下：

- ① **StrAssign(&s, cstr)**: 将字符串常量**cstr**赋给串**s**，即生成其值等于**cstr**的串**s**。
- ② **StrCopy(&s, t)**: 串复制。将串**t**赋给串**s**。
- ③ **StrEqual(s, t)**: 判串相等。若两个串**s**与**t**相等则返回真；否则返回假。
- ④ **StrLength(s)**: 求串长。返回串**s**中字符个数。
- ⑤ **Concat(s, t)**: 串连接:返回由两个串**s**和**t**连接在一起形成的新串。
- ⑥ **SubStr(s, i, j)**: 求子串。返回串**s**中从第**i** ($1 \leq i \leq n$) 个字符开始的、由连续**j**个字符组成的子串。



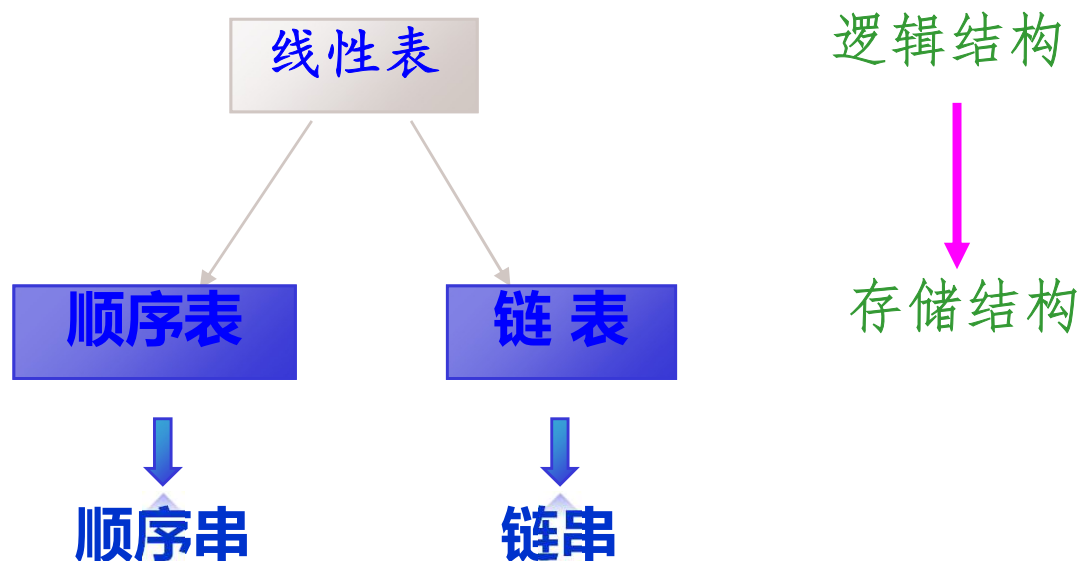
- ⑦ **InsStr(s1, i, s2):** 插入。将串s2插入到串s1的第i ($1 \leq i \leq n+1$) 个字符中, 即将s2的第一个字符作为s1的第i个字符, 并返回产生的新串。
- ⑧ **DelStr(s, i, j):** 删除。从串s中删去从第i ($1 \leq i \leq n$) 个字符开始的长度为j的子串, 并返回产生的新串。
- ⑨ **RepStr(s, i, j, t):** 替换。在串s中, 将第i ($1 \leq i \leq n$) 个字符开始的j个字符构成的子串用串t替换, 并返回产生的新串。
- ⑩ **DispStr(s):** 串输出。输出串s的所有元素值。



4.2 串的存储结构

4.2.1 串的顺序存储及其基本操作实现

串中元素逻辑关系与线性表的相同，串可以采用与线性表相同的存储结构。





对于非紧缩格式（一个char存放一个字符）的顺序串，其类型声明如下：

```
#define MaxSize 100
typedef struct
{   char data[MaxSize];
    int length;
} SqString;
```

← 用来存储字符串

↑ 用来存储字符串长度



顺序串中实现串的基本运算与顺序表的基本运算**类似**。详细算法实现参见第2章**顺序表**部分。

不同点：顺序串参数采用直接传递顺序串的方法，不同于第2章的顺序表算法采用的是顺序表指针。



1) 生成串 **StrAssign(&s, cstr)**

将一个**C/C++**字符串常量**cstr**（以'**\0**'字符标识结尾）赋给顺序串**s**，即生成一个其值等于**cstr**的串**s**。

```
void StrAssign(SqString &s,char cstr[])    //s为引用型参数
{
    int i;
    for (i=0;cstr[i]!='\0';i++)
        s.data[i]=cstr[i];
    s.length=i;                            //设置串s的长度
}
```

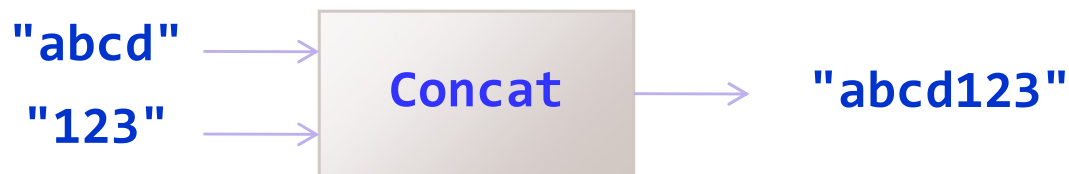
2) 销毁串 **DestroyStr(&s)**

```
void DestroyStr(SqString &s)
{
}
```



6) 串连接Concat(s, t)

返回由两个顺序串s和t连接在一起形成的结果串。



```
SqString Concat(SqString s, SqString t)
{
    SqString str;                //定义结果串
    int i;
    str.length=s.length+t.length;
    for (i=0;i<s.length;i++)      //将s.data[0..s.length-1]复制到str
        str.data[i]=s.data[i];
    for (i=0;i<t.length;i++)      //将t.data[0..t.length-1]复制到str
        str.data[s.length+i]=t.data[i];
    return str;
}
```



【例4.1】设计顺序串上实现串比较运算Strcmp(s, t)的算法。例如：

"ab" < "abcd"

"abcd" < "abd"

解

(1) 比较s和t两个串共同长度范围内的对应字符：

① 若s的字符>t的字符，返回1；

② 若s的字符<t的字符，返回-1；

③ 若s的字符=t的字符，按上述规则继续比较。

(2) 当(1)中对应字符均相同时，比较s和t的长度：

① 两者相等时，返回0；

② s的长度>t的长度，返回1；

③ s的长度<t的长度，返回-1。

"ba">"abc"

"aba"<"bc"

"abc"="abc"

"abc">"ab"

"ab"<"abd"



```
int Strcmp(SqString s, SqString t)
{  int i, comlen;
   if (s.length<t.length)
       comlen=s.length;           //求s和t的共同长度
   else
       comlen=t.length;
   for (i=0;i<comlen;i++)         //在共同长度内逐个字符比较
       if (s.data[i]>t.data[i])
           return 1;
       else if (s.data[i]<t.data[i])
           return -1;
   if (s.length==t.length)        //s==t
       return 0;
   else if (s.length>t.length)    //s>t
       return 1;
   else return -1;                //s<t
}
```

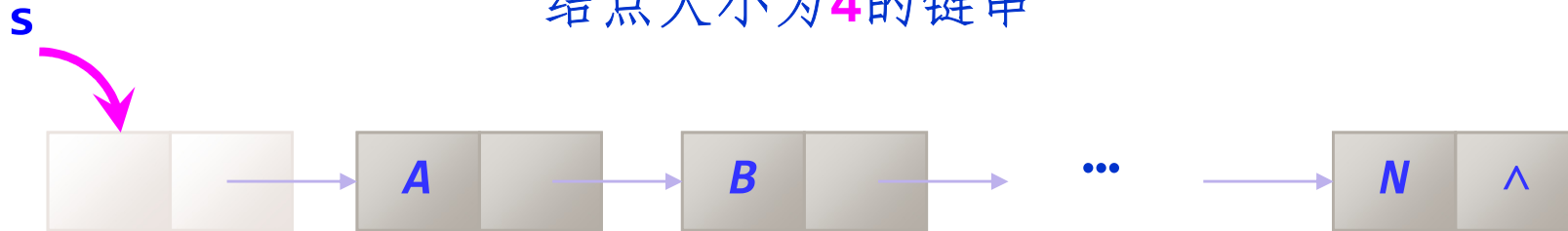


4.2.2 串的链式存储及其基本操作实现

- 链串的组织形式与一般的链表类似。
- 链串中的一个结点可以存储多个字符。通常将链串中每个结点所存储的字符个数称为**结点大小**。



结点大小为**4**的链串

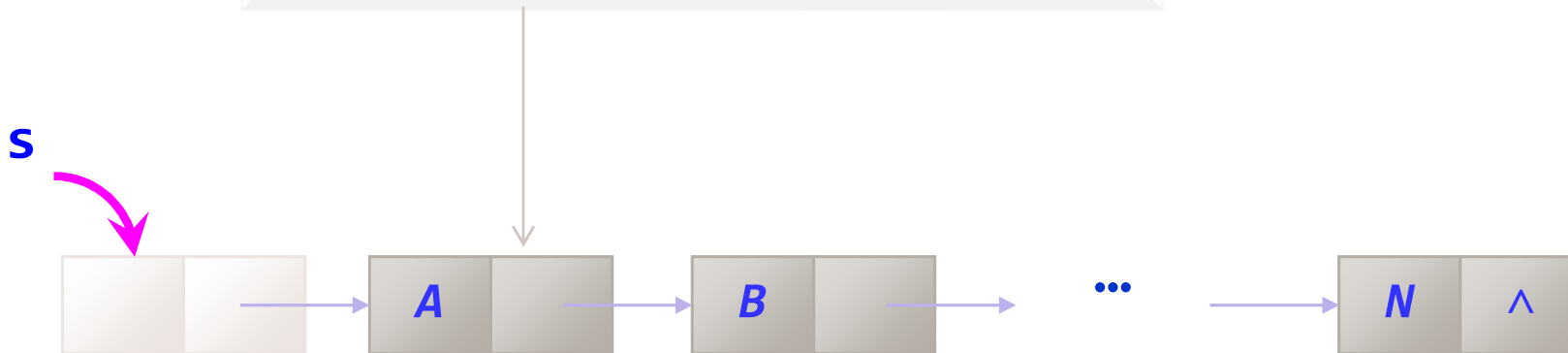


结点大小为**1**的链串



链串结点大小1时，链串的结点类型声明如下：

```
typedef struct snode
{   char data;
    struct snode *next;
} LinkStrNode;
```





链串中实现串的基本运算与单链表的基本运算类似。



1) 生成串StrAssign(&s, cstr)

将一个C/C++字符串常量**cstr**（以'\0'字符标识结尾）赋给链串**s**，即生成一个其值等于**cstr**的链串**s**。

```
void StrAssign(LinkStrNode *&s, char cstr[])
{
    int i;
    LinkStrNode *r, *p;
    s = (LinkStrNode *)malloc(sizeof(LinkStrNode));
    r = s;                                     //r始终指向尾结点
    for (i = 0; cstr[i] != '\0'; i++)
    {
        p = (LinkStrNode *)malloc(sizeof(LinkStrNode));
        p->data = cstr[i];
        r->next = p; r = p;
    }
    r->next = NULL;                            //尾结点next域置为空
}
```



2) 销毁串DestroyStr(&s)

该运算和销毁带头结点单链表运算的实现过程相同。

```
void DestroyStr(LinkStrNode *&s)
{
    LinkStrNode *pre=s,*p=s->next;    //pre指向结点p的前驱结点
    while (p!=NULL)                    //扫描链串s
    {
        free(pre);                     //释放pre结点
        pre=p;                          //pre、p同步后移一个结点
        p=p->next;
    }
    free(pre);
}
```



6) 串连接Concat(s, t)

求由两个链串s和t的数据结点连接在一起形成结果串。

```
LinkStrNode *Concat(LinkStrNode *s, LinkStrNode *t)
{
    LinkStrNode *str, *p=s->next, *q, *r;
    str=(LinkStrNode *)malloc(sizeof(LinkStrNode));
    r=str;                                //r指向结果串的尾结点
    while (p!=NULL)                       //用p扫描s的所有数据结点
    {
        q=(LinkStrNode *)malloc(sizeof(LinkStrNode));
        q->data=p->data;                  //将p结点复制到q结点中
        r->next=q; r=q;                  //将q结点链接到str的末尾
        p=p->next;
    }
    p=t->next;
    while (p!=NULL)                       //用p扫描t的所有数据结点
    {
        q=(LinkStrNode *)malloc(sizeof(LinkStrNode));
        q->data=p->data;                  //将p结点复制到q结点中
        r->next=q; r=q;                  //将q结点链接到str的末尾
        p=p->next;
    }
    r->next=NULL;                         //尾结点的next域置为空
    return str;
}
```



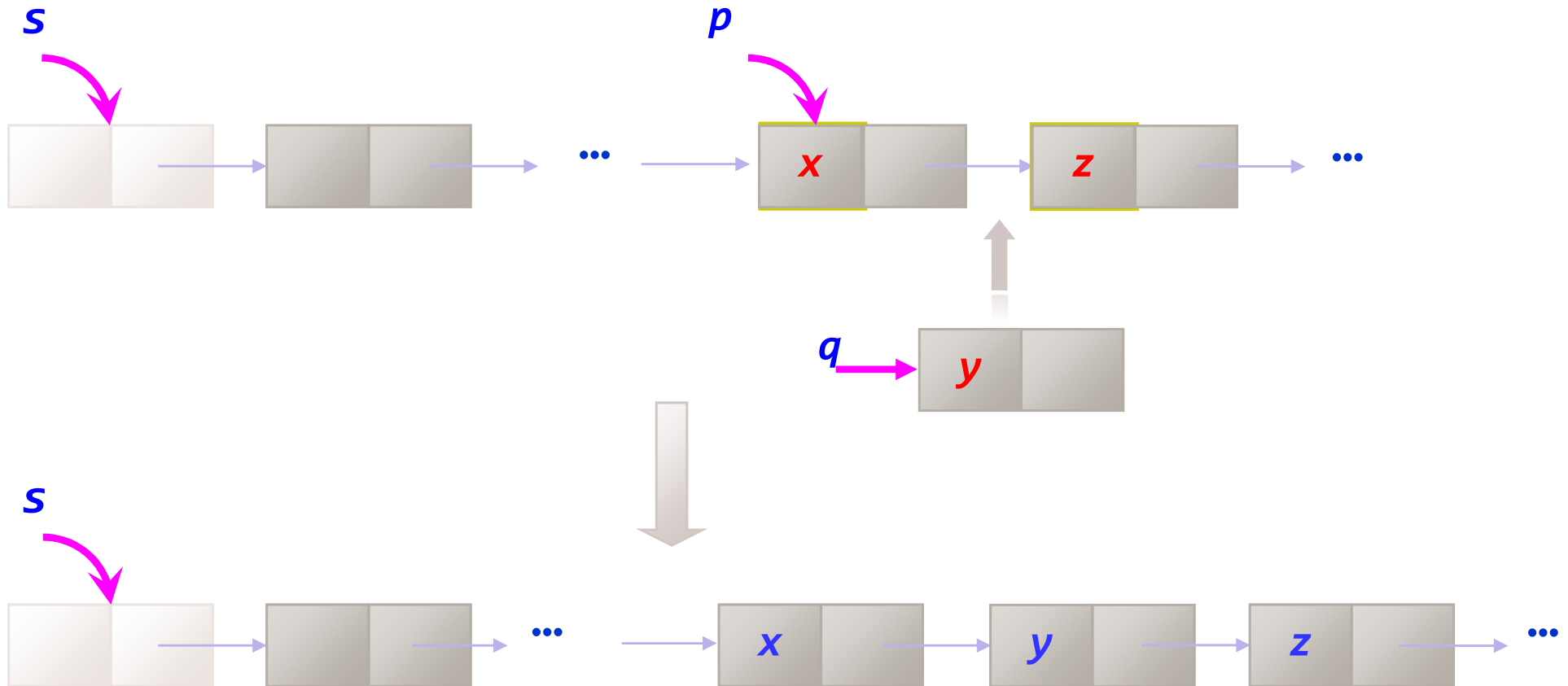
【例4.3】 在链串中，设计一个算法把最先出现的子串“*ab*”改为“*xyz*”。

解

① 查找： $p \rightarrow \text{data} = 'a' \quad \&\& \quad p \rightarrow \text{next} \rightarrow \text{data} = 'b'$



② 替换





```
void Repl(LinkStrNode *&s)
{
    LinkStrNode *p=s->next, *q;
    int find=0;
    while (p->next!=NULL && find==0)           //查找ab子串
    {
        if (p->data=='a' && p->next->data=='b')
        {
            p->data='x'; p->next->data='z';
            q=(LinkStrNode *)malloc(sizeof(LinkStrNode));
            q->data='y'; q->next=p->next; p->next=q;
            find=1;
        }
        else p=p->next;
    }
}
```

算法的时间复杂度为 $O(n)$ 。

替换为xyz



4.3 串的模式匹配

4.3.1 Brute-Force算法

目标串 s

是子串吗?

模式串 t

模式匹配

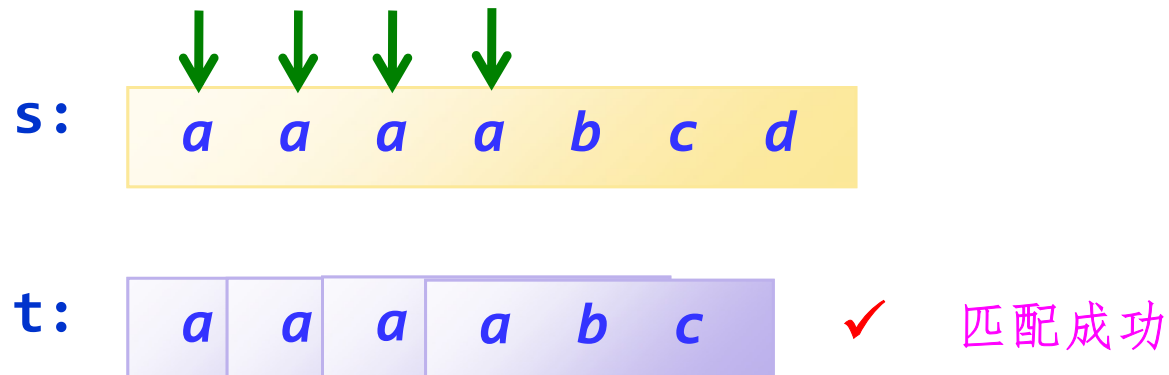
- **成功**是指在目标串 s 中找到一个模式串 t —— t 是 s 的子串，返回 t 在 s 中的位置。
- **不成功**则指目标串 s 中不存在模式串 t —— t 不是 s 的子串，返回 -1。



Brute-Force 简称为 **BF算法**，亦称简单匹配算法。采用穷举的思路。

BF是指暴力的意思！

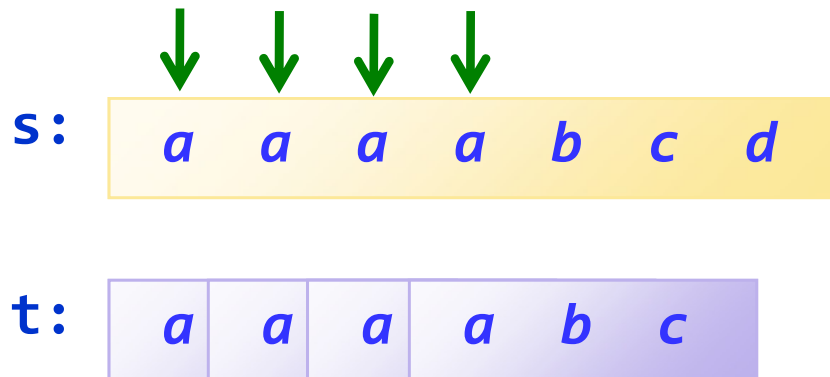
例如， $s = \text{"aaaabcd"}$ ， $t = \text{"abc"}$ 。





BF算法思路：从s的每一个字符开始依次与t的字符进行匹配。

从s₀开始匹配第1趟第2趟第3趟第4趟 过程



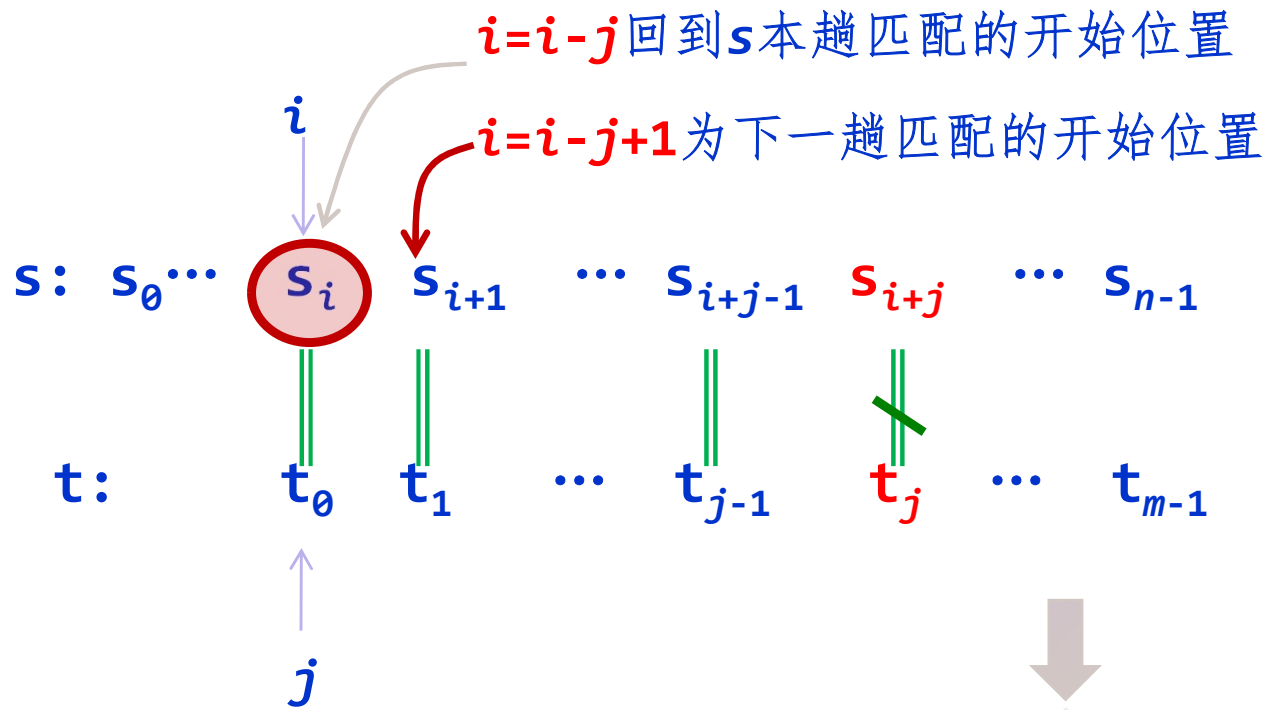
- i, j 分别扫描字符串 s 和 t
- i 从 0 到 $s.length - t.length$ 循环。
- 对于每个 $s[i]$, $k=i, j=0$ 开始比较
- 如果 t 扫描完毕, 则返回 i
- 若 s 全部比较完毕都没有返回, 则返回 -1



```
int index(SqString s,SqString t)
{  int i,j,k;
   for (i=0;i<=s.length-t.length;i++)
   {  for (k=i,j=0; k<s.length && j<t.length &&
        s.data[k]==t.data[j]; k++,j++);
      if (j==t.length)
        return(i);
    }
   return(-1);
}
```



实际上，可以只采用两个变量 i 、 j ，通过 i 回退实现：



从 s_i 开始匹配：

- 若 $s_i = t_j \Rightarrow i++, j++$
- 若 $s_i \neq t_j \Rightarrow i = i - j + 1, j = 0$



对应的BF算法如下：

```
int BF(SqString s, SqString t)
{  int i=0, j=0;
  while (i<s.length && j<t.length)
  {  if (s.data[i]==t.data[j])
      {  i++;
          j++;
          //主串和子串依次匹配下一个字符
      }
      else
          //主串、子串指针回溯重新开始下一次匹配
          {  i=i-j+1;
              j=0;
              //主串从下一个位置开始匹配
              //子串从头开始匹配
          }
  }
  if (j>=t.length)
      return(i-t.length);
  else
      return(-1);
  //或者if (j==t.length)
  //返回匹配的第一个字符的下标
  //模式匹配不成功
}
```



BF算法分析

- 算法在字符比较不相等，需要回溯（即 $i=i-j+1$ ）：即退到 s 中的下一个字符开始进行继续匹配。
- 最好情况下的时间复杂度为 $O(m)$ 。
- 最坏情况下的时间复杂度为 $O(n \times m)$ 。
- 平均的时间复杂度为 $O(n \times m)$ 。



示例

假设串采用顺序串存储结构。设计一个算法求串 t 在串 s 中出现的次数，如果不是子串返回0。采用BF算法求解。

例如"aa"在"aaab"中出现2次。

解

- 用**count**累计 t 在串 s 中出现的次数（初始值为0）。
- 采用BF方法，在 s 中找到子串 t 后不是退出，而是**count**增加1， i 置为 k 并继续查找，直到整个字符串查找完毕。



```
int StrCount1(SqString s,SqString t)
//利用BF算法求t在s中出现的次数
{  int i=0,j,k,count=0;
    while (i<=s.length-t.length)
    {  for (k=i,j=0; k<s.length && j<t.length &&
        s.data[k]==t.data[j]; k++,j++);
        if (j==t.length)                //找到一个子串
        {  count++;                      //累加次数
            i=k;                          //i从j开始
        }
        else i++;                        //i增加1
    }
    return count;
}
```



4.3.2 KMP算法

KMP算法是D.E.Knuth、J.H.Morris和V.R.Pratt共同提出的，简称**KMP算法**。

该算法较BF算法有较大改进，主要是**消除了主串指针的回溯**，从而使算法效率有了某种程度的提高。



KMP算法的思路

- 基于**BF**算法，采用空间换时间的方式，提取保存有利于匹配的信息。
- 提取**s**还是**t**中的信息？每次从**s**不同字符开始匹配，而**t**总是从**t₀**开始匹配 → 提取**t**中的信息
- **t**中的什么信息？ → 部分匹配信息。



开始匹配的字符

下次开始匹配的字符

s: 0 1 2 3 4 5
a a a a a b

t: a a a b a b

下次匹配的两个字符

从t中发现: b前面有2个字符和开头的2个字符相同

t中每个位置都有这种信息
用一个数组next保存，如next[3]=2



1

$\text{next}[j]$ 是指 t_j 字符前有多少个字符与 t 开头的字符相同

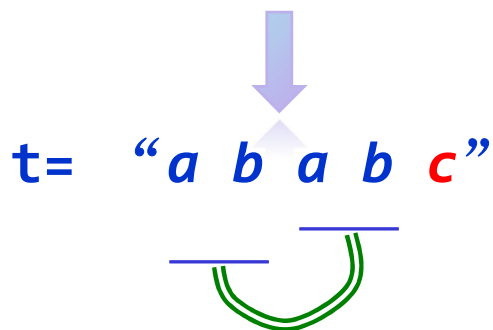
模式串 t 中字符 t_j , 存在某个 k ($0 < k < j$), 使得以下成立:

$$t_0 t_1 \cdots t_{k-1} = t_{j-k} t_{j-k+1} \cdots t_{j-1}$$

开头的 k 个字符

t_j 前面的 k 个字符

例如, $t =$ “a b a b c” 考虑 $t[4]='c'$



⇒ $k=2$, 所以 $\text{next}[4] = k = 2$ 。



2

$\text{next}[j]$ 是指 t_j 字符前最多有多少个字符与 t 开头的字符相同

模式串 t 中字符 t_j , 存在某个 k ($0 < k < j$), 使得以下成立:

$$t_0 t_1 \cdots t_{k-1} = t_{j-k} t_{j-k+1} \cdots t_{j-1}$$

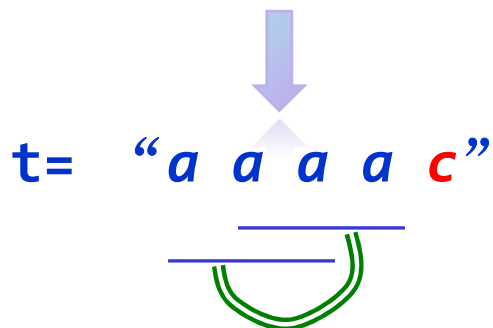
开头的 k 个字符

t_j 前面的 k 个字符

求最大的 k

0 1 2 3 4

例如, $t = \text{"a a a a c"}$ 考虑 $t[4] = \text{'c'}$



$\Rightarrow k=3$, 所以 $\text{next}[4] = k = 3$ 。



3

t_j 字符前面的子串最多从 t_1 开始的, 不含 t_0

模式串 t 中字符 t_j , 存在某个 k ($0 < k < j$), 使得以下成立:

$$t_0 t_1 \cdots t_{k-1} = t_{j-k} t_{j-k+1} \cdots t_{j-1}$$

开头的 k 个字符

t_j 前面的 k 个字符

这个字符不能是 t_0

0 1 2

例如, $t = \text{"a a c"}$ 考虑 $t[2] = \text{'c'}$

$t = \text{"a a c"}$

⇒ $k=1$, 所以 $\text{next}[2] = k = 1$ 。



4

$\text{next}[0]$ 的特殊含义

$\text{next}[0]$ 设置为 -1

因为 s 的字符 s_i 与 t_0 不相同，没有任何有用的部分匹配信息，直接从下一趟 (s_{i+1}/t_0) 开始匹配。



对于模式串 $t = "t_0 t_1 \cdots t_j \cdots t_{m-1}"$

归纳起来，定义 $next[j]$ 数组如下：

$$next[j] = \begin{cases} \text{MAX}\{ k \mid 0 < k < j, \text{ 且 } \overbrace{t_0 t_1 \cdots t_{k-1}}^{\text{开头的 } k \text{ 个字符}} = \overbrace{t_{j-k} t_{j-k+1} \cdots t_{j-1}}^{t_j \text{ 前面的 } k \text{ 个字符}} \} & \text{当此集合非空时} \\ -1 & \text{当 } j=0 \text{ 时} \\ 0 & \text{其他情况} \end{cases}$$



$t = \text{"aaab"}$ 对应的next数组如下:

j	0	1	2	3
$t[j]$	a	a	a	b
$next[j]$	-1	0	1	2

$$t_0 = t_1 = "a" \quad t_2 = "aa"$$



由模式串 t 求next值的算法:

```
void GetNext(SqString t, int next[])
{   int j, k;
    j=0; k=-1; next[0]=-1;
    while (j<t.length-1)
    {   if (k==-11 || t.data[j]==t.data[k])
        {   j++; k++;
            next[j]=k;
        }
    }
}
```

由next[j]求next[j+1], 所以j取值 $0 \sim m-2$

求出next[3]=2, k=2

0 1 2 3 4
t= "a a a a c"
j=3, k=2, 所指字符相同

j++, k++
next[4]=3

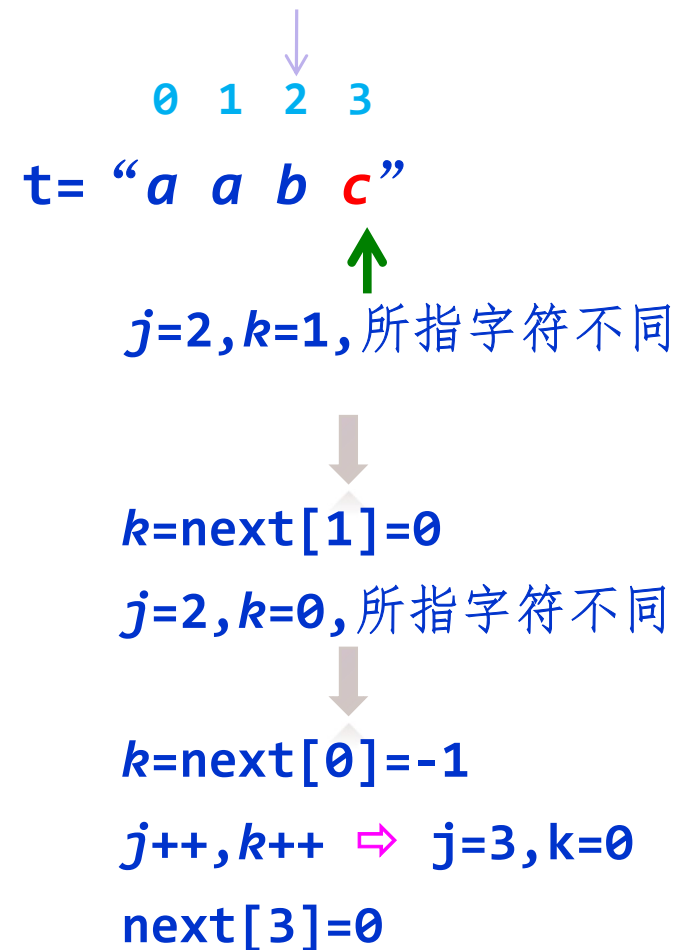


由模式串t求next值的算法:

```
void GetNext(SqString t, int next[])
{  int j, k;
   j=0; k=-1; next[0]=-1;

   while (j<t.length-1)
   {  if (k==-1 || t.data[j]==t.data[k])
      {  j++; k++;
         next[j]=k;
      }
      else
         k=next[k];
   }
}
```

求出next[2]=1, k=1





KMP过程

```
i=0; j=0;
while (s和t都没有扫描完)
{  if (j=-1或者它们所指字符相同)
    i和j分别增1;
    else
        i不变, j回退到j=next[j] (即模式串右滑);
}
if (j超界)
    返回i;                                //模式匹配成功
else
    返回-1;                                //模式匹配失败
```



KMP算法:

```
int KMPIndex(SqString s, SqString t)
{  int next[MaxSize], i=0, j=0;
   GetNext(t, next);
   while (i<s.length && j<t.length)
   {
       if (j==-1 || s.data[i]==t.data[j])
       {   i++;
           j++;                               //i、j各增1
       }
       else j=next[j];                       //i不变, j后退
   }
   if (j>=t.length)
       return(i-t.length);                   //返回匹配模式串的首字符
   else
       return -1;                           //返回不匹配标志
}
```



KMP算法分析

- 设串 s 的长度为 n ，串 t 长度为 m 。
- KMP算法中求next数组的时间复杂度为 $O(m)$ ，在后面的匹配中因主串 s 的下标不减即不回溯，比较次数可记为 n ，所以KMP算法平均时间复杂度为 $O(n+m)$ 。
- 最坏的时间复杂度为 $O(n \times m)$ 。



示例

已知字符串 S 为“ $abaabaabacacaabaabcc$ ”，模式串 t 为“ $abaabc$ ”，采用KMP算法进行匹配，第一次出现“失配”($s[i] \neq t[j]$)时， $i=j=5$ ，则下次开始匹配时， i 和 j 的值分别是（ ）。

- A. $i=1, j=0$ B. $i=5, j=0$ C. $i=5, j=2$ D. $i=6, j=2$

说明：本题为2015年全国考研题

j	0	1	2	3	4	5
$t[j]$	a	b	a	a	b	c
$next[j]$	-1	0	0	1	1	2

选C



示例

假设串采用顺序串存储结构。设计一个算法求串 t 在串 s 中出现的次数，如果不是子串返回 0 。采用KMP算法求解。

例如" aa "在" $aaaab$ "中出现 2 次。

解

- 用 **count** 累计 t 在串 s 中出现的次数（初始值为 0 ）。
- 采用 **KMP** 方法，当匹配成功时，**count** 增加 1 ，并且置 j 为 0 重新开始比较。

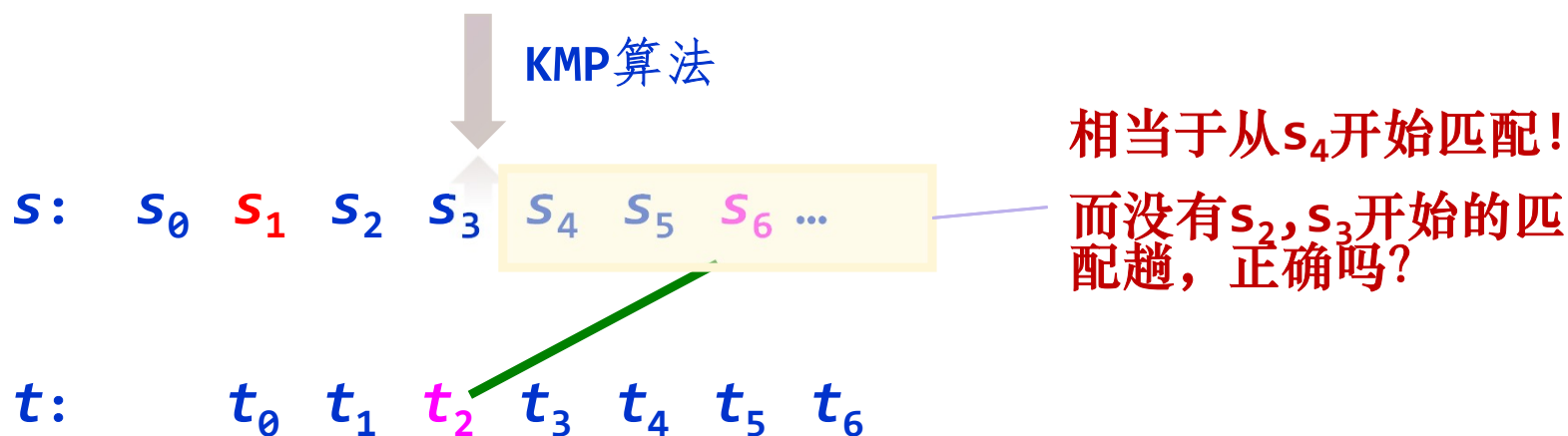
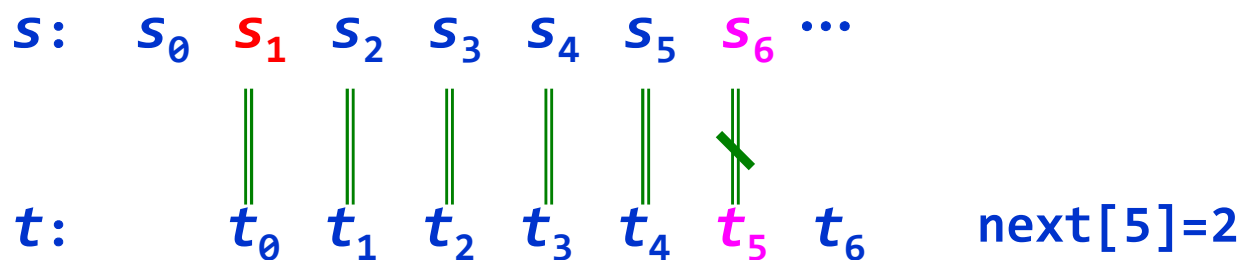


```
int StrCount2(SqString s,SqString t)
//利用KMP算法求t在s中出现的次数
{  int i=0,j=0,count=0;
    int next[MaxSize];
    GetNext(t,next);
    while (i<s.length && j<t.length)
    {  if (j==-1 || s.data[i]==t.data[j])
        {  i++;
            j++;
            //i, j各增1
        }
        else
        {  j=next[j];
            //i不变, j后退
            //成功匹配1次
            if (j>=t.length)
            {  count++;
                j=0;
                //j设置为0, 继续匹配
            }
        }
    }
    return count;
}
```



KMP算法的正确性说明

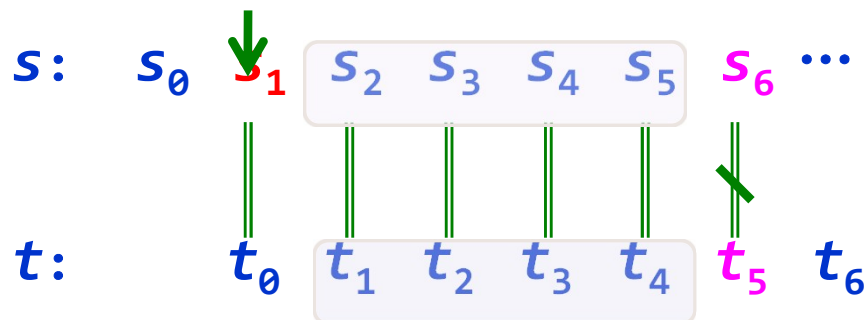
设目标串 $s = "s_0s_1s_2s_3s_4s_5s_6..."$ ，模式串 $t = "t_0t_1t_2t_3t_4t_5"$ ， $\text{next}[5]=2$ 。
从 s_1 开始匹配（第2趟），失配处为 $s_6 \neq t_5$ ，这里 $i=6$ ， $j=5$ ， $k=2$ 。





考虑 s_2 开始的匹配趟

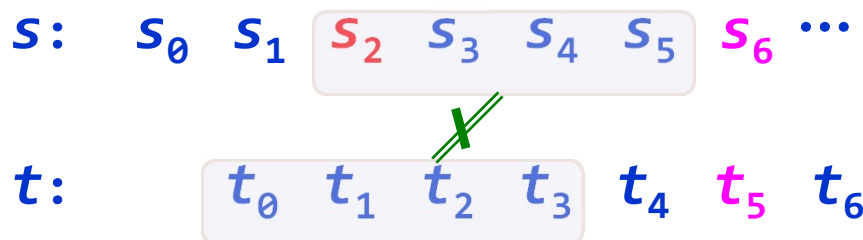
$\text{next}[5]=2$



" $s_2s_3s_4s_5$ " = " $t_1t_2t_3t_4$ "

$\text{next}[5]=2 \Rightarrow$ " t_3t_4 " = " t_0t_1 ", 而 " $t_1t_2t_3t_4$ " $\neq$ " $t_0t_1t_2t_3$ "

" $s_2s_3s_4s_5$ " $\neq$ " $t_0t_1t_2t_3$ "

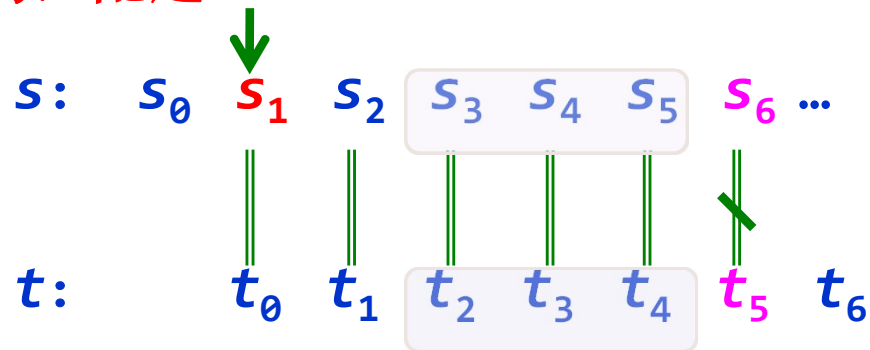


从 s_2 开始的匹配趟是不必要的



考虑 s_3 开始的匹配趟

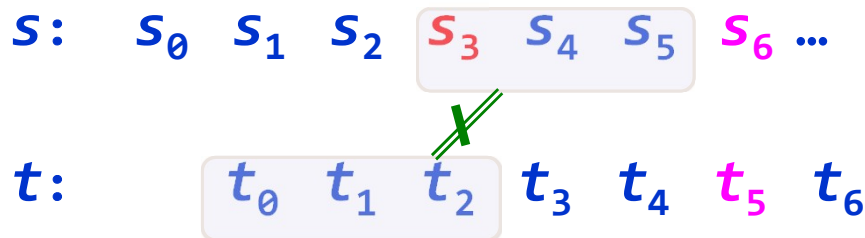
$\text{next}[5]=2$



" $s_3s_4s_5$ " = " $t_2t_3t_4$ "

$\text{next}[5]=2 \Rightarrow$ " t_3t_4 " = " t_0t_1 ", 而 " $t_2t_3t_4$ " $\neq$ " $t_0t_1t_2$ "

" $s_3s_4s_5$ " $\neq$ " $t_0t_1t_2$ "



从 s_3 开始的匹配趟是不必要的



KMP算法的改进



示例

设目标串 $s = \text{"aaabaaaaab"}$ ，模式串 $t = \text{"aaaab"}$ 。KMP模式匹配过程。

(1) 求 t 的 next

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

	0	1	2	3	4	5	6	7	8
s:	a	a	a	b	a	a	a	a	b
t:	a	a	a	a	b				
	0	1	2	3	4				

Diagram showing string matching between s and t. Pink arrows indicate matches at indices 0, 1, and 2. At index 3, a pink arrow points from s[3] (b) to t[3] (a), which is crossed out with a pink slash, indicating a mismatch.

失败:

i=3

j=3, j=next[3]=2



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

$i=3$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a **b** a a a a b

$j=2$ $t:$ a a **a** a b

 0 1 2 3 4

失败:

$i=3$

$j=2, j=\text{next}[2]=1$



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

$i=3$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a **b** a a a a b

$j=1$ $t:$ a **a** a a b

 0 1 2 3 4

失败:

$i=3$

$j=1, j=\text{next}[1]=0$



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

$i=3$
 $j=0$

s: 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

t: a a a a b
 0 1 2 3 4

失败:

$i=3$

$j=0, j=\text{next}[0]=-1$



j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

因为 $j = -1$:

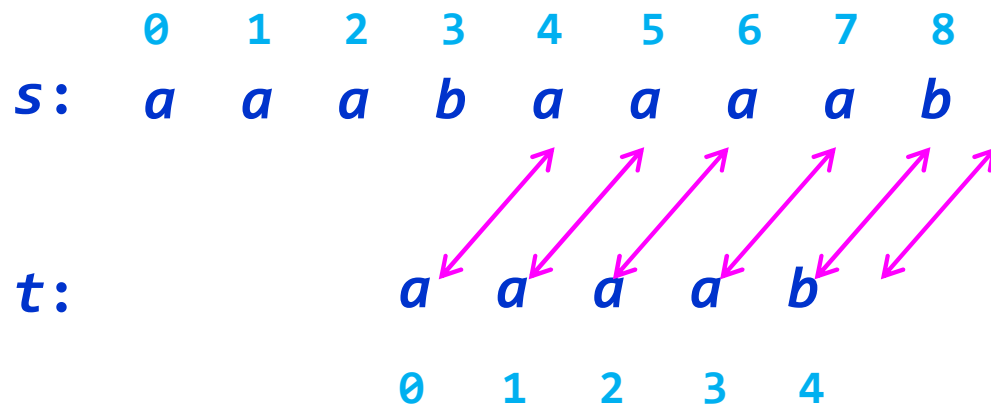
$i++$;

$j++$;

即:

$i = 4$

$j = 0$



成功匹配:

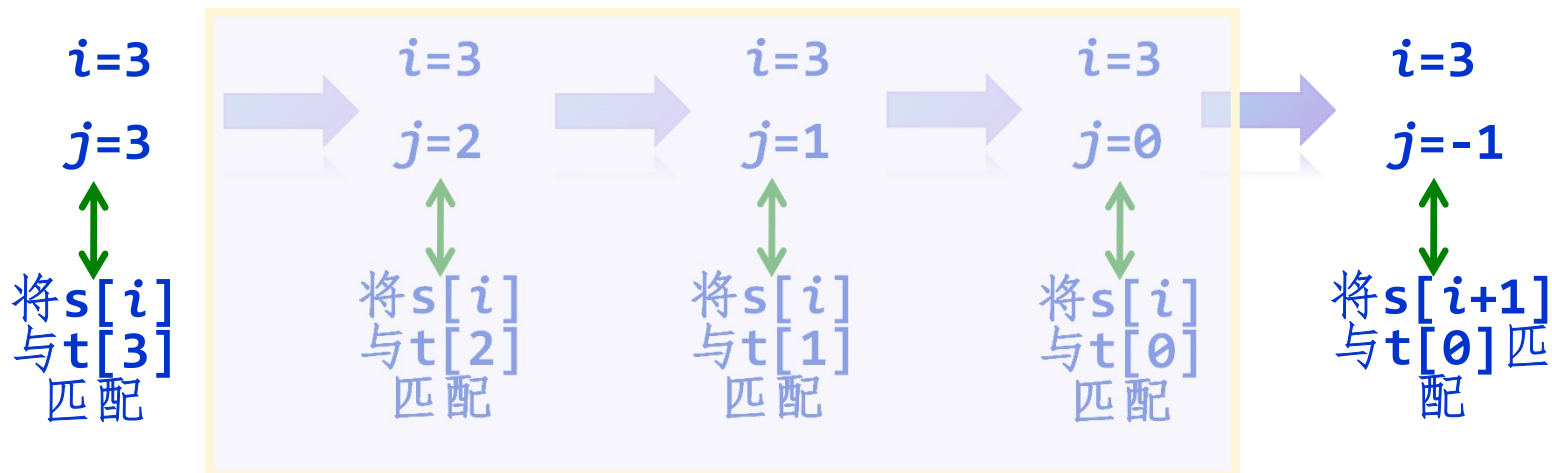
返回 $i - t.length$

$= 9 - 5 = 4$



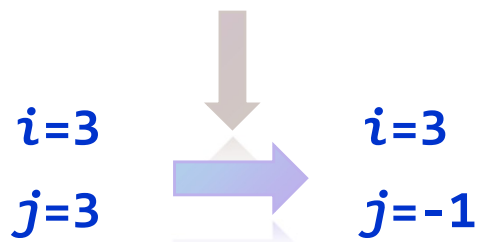
j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3

前面的匹配过程：



是不必要的

因为 $t[3]=t[2]=t[1]=t[0]='a'$





将next改为nextval:

j	0	1	2	3	4
t[j]	a	a	a	a	b
next[j]	-1	0	1	2	3
nextval[j]	-1	-1	-1	-1	3

next[1]=0

t[1]=t[next[1]]=t[0]='a'

∴ nextval[1]=nextval[0]=-1

t[4]='b' ≠ t[next[4]]=t[3]='a'

∴ nextval[4]=next[4]



- nextval[0]=-1
- 当t[j]=t[next[j]]时: nextval[j]=nextval[next[j]]
- 否则: nextval[j]=next[j]

用nextval取代next, 得到改进的KMP算法。



使用改进后的KMP算法示例：

j	0	1	2	3	4
t[j]	a	a	a	a	b
nextval[j]	-1	-1	-1	-1	3

	0	1	2	3	4	5	6	7	8
s:	a	a	a	b	a	a	a	a	b
	↑↓	↑↓	↑↓						
t:	a	a	a	a	b				
	0	1	2	3	4				

失败：

i=3

j=3, j=nextval[3]=-1



j	0	1	2	3	4
t[j]	a	a	a	a	b
nextval[j]	-1	-1	-1	-1	3

因为 $j = -1$:

$i++;$

$s:$ 0 1 2 3 4 5 6 7 8
 a a a b a a a a b

$j++;$

$t:$

 a a a a b
 0 1 2 3 4

即

成功匹配:

$i = 4$

返回 $i - t.length$

$j = 0$

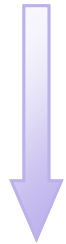
$= 9 - 5 = 4$

改进后的KMP算法进一步提高模式匹配的效率。



数据结构经典算法的启示

BF 算法



利用模式串中部分匹配信息

KMP 算法

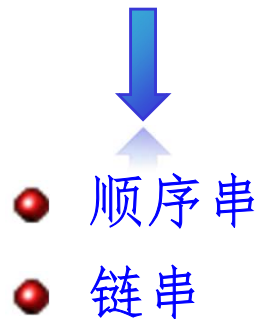


第4章 小结

1

串的存储结构

① 串是一种特殊的线性表，是线性表的一个子集。





② 链串只能采用单链表吗？

- 不一定。需要根据需要情况而定。
- 如果需要从某个结点出发前后查找，可以采用双链表。
- 如果需要快速查找尾结点，可以采用循环双链表。



2

串的算法设计

① 串的基本算法设计

借鉴线性表的算法设计方法。

- 顺序串 ⇔ 顺序表
- 链 串 ⇔ 单链表



② 串的模式匹配算法设计

- BF 算法
- KMP 算法



- @ 为什么**KMP**算法平均性能更高？
- @ 是不是任何情况下**KMP**算法都好于**BF**算法？



示例

假设串采用字符数组存储。修改BF算法在一个字符串 s 中查找子串 t 出现次数。例如，对于 $s = \text{"abcababcd"}$ ， $t = \text{"abcab"}$ 。这里认为 t 在 s 中出现2次（考虑重叠部分）。

解

- 用 **count** 记录 t 在 s 中出现的次数。
- 设 s 、 t 串的长度分别为 m 、 n 。用 i 、 j 方便扫描设 s 、 t 串。修改 BF 算法，当匹配成功时 i 不跳过 n 个字符，而是从下一个位置开始继续比较，这样达到重复匹配的目的。

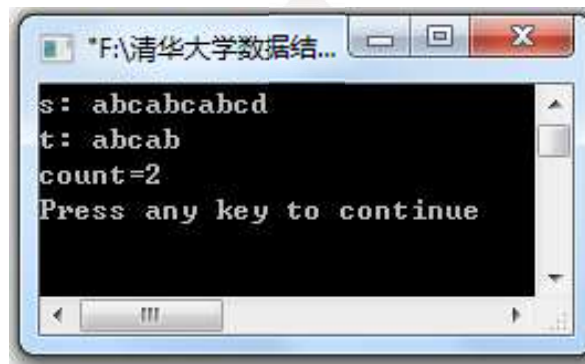


算法如下：

```
int Substrcount1(char *s,char *t)    //利用BF算法求t在s中出现的次数
{  int i,j,k,count=0;
    int m=strlen(s);                //求出s的长度m
    int n=strlen(t);                //求出t的长度n
    for (i=0;i<=m-n;i++)
    {  for (k=i,j=0;k<m && j<n && s[k]==t[j];k++,j++);
        if (j==n)                    //j等于子串t的长度
            count++;
    }
    return (count);
}
```



```
void main()
{  char s[]="abcabcabcd";
   char t[]="abcab";
   printf("s: %s\n",s);
   printf("t: %s\n",t);
   printf("count=%d\n",Substrcount1(s,t));
}
```



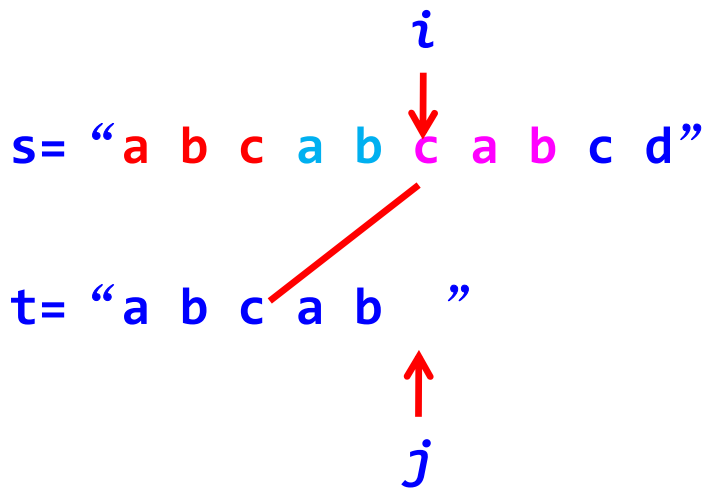


示例

假设串采用字符数组存储。修改KMP算法在一个字符串 s 中查找子串 t 出现次数。例如，对于 $s = \text{"abcabcbcd"}$ ， $t = \text{"abcab"}$ 。这里认为 t 在 s 中出现2次（考虑重叠部分）。

解

用 count 记录 t 在 s 中出现的次数。修改KMP算法匹配过程。 例如：



当匹配成功时， $j=5$ ，设置 $\text{next}[5]=2$ ，让 $s[i]$ 与'c'字符开始比较！



在KMP算法中，除了求 $t_0 \sim t_{n-1}$ （含 n 个字符）的 $\text{next}[j]$ ，还要求出 t_n 的 $\text{next}[n]$ 值，以便实现重复匹配。

$t = \text{"a b c a b"} , n=5$

j	0	1	2	3	4	5
$t[j]$	a	b	c	a	b	
$\text{next}[j]$	-1	0	0	0	1	2



```
void GetNext1(char *t, int next[]) //由模式串t求出next值
{
    int j, k;
    int n=strlen(t);
    j=0; k=-1; next[0]=-1;
    while (j<n) //改为需要求next[n]
    {           //k为-1或比较的字符相等时
        if (k==-1 || t[j]==t[k])
        {
            j++;k++;
            next[j]=k;
        }
        else k=next[k];
    }
}
```



```
int Substrcount2(char *s, char *t) //利用KMP算法求t在s中出现的次数
{
    int next[MaxSize], i=0, j=0, count=0;
    GetNext1(t, next);
    while (s[i] && t[j])
    {
        if (j==-1 || s[i]==t[j])
        {
            i++;
            j++;
            //i, j各增1
        }
        else j=next[j];
        //i不变, j后退
    }
    if (!t[j])
    {
        count++;
        j=next[j];
        //改为将j设置为next[j], 继续匹配
    }
}
return count;
}
```



```
void main()  
{ char s[]="abcabcabcd";  
  char t[]="abcab";  
  printf("s: %s\n",s);  
  printf("t: %s\n",t);  
  printf("count=%d\n",Substrcount2(s,t));  
}
```

A screenshot of a Windows command prompt window. The title bar reads 'F:\清华大学数据结...'. The window contains the following text:

```
s: abcabcabcd  
t: abcab  
count=2  
Press any key to continue
```



实验练习

以下两类2选1即可:

➤ **类别1**: P135, 练习题4, 题目: 4、5、8; P136, 上机实验题4, 题号: 实验题3; P138, 题号: 4.leetcode28-实现 strStr ()

➤ **类别2**:

✓ Leetcode, 题号: 3498、2315、2351、2278、2390